

PARTE B

Prueba Técnica de Selección

1. Propósito de la prueba

Esta prueba está diseñada para validar si un candidato puede trabajar en un programa real de transformación de automatización empresarial, donde el reto no es “hacer un bot/rpa”, sino decidir qué se migra, qué se consolida, qué se reemplaza, como se evoluciona y cómo se demuestra valor sin depender de ambientes del cliente.

La prueba evalúa:

- Criterio para migrar desde una plataforma de RPAs hacia una arquitectura evolutiva moderna.
- Capacidad para construir soluciones locales reproducibles y demostrables.
- Uso correcto de componentes reutilizables y reducción de fragmentación.
- Manejo de errores, trazabilidad, pruebas y rollback.
- Comunicación técnica clara frente a negocio, arquitectura y operación.

No se busca evaluar memorización de herramientas, sino criterio de diseño, ejecución y socialización.

2. Restricción clave

No se entregarán ambientes, credenciales, datos productivos ni infraestructura del cliente. Cada candidato debe desarrollar su solución en su propio entorno local y demostrarla en el espacio de socialización.

Lo que sí puede usar

- Python, Java
- Docker o Docker Compose, si lo considera útil.
- Bases locales como SQLite, archivos planos, CSV, Excel, JSON o PostgreSQL local.
- Datos sintéticos o anonimizados.
- Librerías open source.
- Se suministra un archivo base de datos de taskbot de ejemplo, lo puedes usar o generar tu propia base.

Lo que no se permitirá como dependencia crítica

- Ambientes corporativos preconfigurados.
- Credenciales del cliente.
- Conexiones obligatorias a servicios externos sin alternativa local.
- Suposiciones de acceso a Appian, n8n o Power Platform reales para completar la prueba.

Si el candidato desea mapear su propuesta a esas tecnologías, puede hacerlo en el documento, pero la demostración debe correr localmente.

3. Formato de entrega

El candidato debe entregar un único paquete con tres componentes:

1. Repositorio o carpeta ejecutable localmente

- Código fuente.
- Instrucciones de ejecución.
- Archivos de configuración.
- Datos de ejemplo.

2. Documento breve de diseño

- Arquitectura propuesta.
- Decisiones tecnológicas.
- Riesgos y mitigaciones.
- Plan de pruebas.

3. Demostración en socialización

- Ejecución local en vivo.
- Explicación del flujo.
- Evidencia de trazabilidad, pruebas y salida esperada.

Entregables mínimos obligatorios

- README con pasos exactos para correr la solución.
- Diagrama de arquitectura o flujo.
- Evidencia de pruebas ejecutadas.
- Explicación de rollback o recuperación ante fallo.
- Resumen ejecutivo de máximo 1 página.
- Código fuente local.
- Un flujo de n8n exportable o documentado.
- Reglas de clasificación o prompts del agente.
- Inventario de ejemplo.
- Reporte de salida con recomendaciones.

Nombre del caso

Agente de inteligencia para consolidación y priorización de migración de taskbots

Este caso mide algo distinto a la automatización tradicional: capacidad de diseño, criterio de consolidación y uso práctico de n8n o de un agente local para convertir una masa de automatizaciones fragmentadas en decisiones accionables.

Escenario

El candidato debe construir una solución local que reciba un inventario simplificado de taskbots y genere una recomendación de migración y consolidación.

El inventario puede incluir, por ejemplo:

- Nombre del taskbot.
- Propósito funcional.
- Aplicaciones involucradas.
- Tipo de interacción: API, archivo, email, BD, UI legacy.
- Frecuencia de ejecución.
- Riesgo operacional.
- Dependencias conocidas.
- Evidencia de duplicidad o similitud con otros taskbots.

Objetivo funcional

La solución debe identificar automáticamente:

1. Qué taskbots son candidatos a consolidación.
2. Qué taskbots pueden moverse a n8n, un BPM, Power Platform, microservicio o RPA selectivo o automatización a la medida en Python o Java.
3. Qué componentes reutilizables podrían extraerse.
4. Qué dependencias o riesgos requieren revisión manual.
5. Qué ola de migración sugerida recibe cada caso.

Forma de implementación esperada (no obligatoria)

El candidato puede implementarlo en Python o Java, con n8n como orquestador o con un agente local procode si lo prefiere.

Una implementación sólida podría tener esta forma:

- **n8n** como orquestador principal.
- **Servicio local en Python o Java** para análisis, clasificación y scoring.
- **Archivo CSV, JSON o SQLite** como fuente de inventario.
- **Reglas + heurísticas + agente** para clasificar casos.
- **Salida en JSON y reporte HTML/Markdown** con las recomendaciones.

Flujo mínimo esperado

1. Cargar un inventario local de taskbots.
2. Normalizar descripciones y atributos.
3. Detectar similitud o duplicidad entre taskbots.
4. Clasificar destino tecnológico sugerido.
5. Asignar una prioridad de migración.
6. Generar un reporte con hallazgos y recomendaciones.
7. Registrar trazabilidad y errores.

Resultado que debe mostrar en la demo

El candidato debe poder demostrar en vivo algo como esto:

- “Estos 12 taskbots son variantes de 3 utilidades reutilizables.”
- “Estos 8 casos son candidatos a n8n por ser integración API-first.”
- “Estos 5 deben quedar en RPA selectivo porque dependen de UI legacy.”
- “Este grupo debería convertirse en un microservicio compartido.”
- “Esta lista entra en ola 1 por valor alto y complejidad baja.”

Qué se evalúa aquí

- Capacidad de innovación aplicada, no solo ejecución mecánica.
- Buen uso de n8n como orquestador.
- Capacidad para combinar reglas determinísticas con criterio asistido por agente.
- Diseño local reproducible.
- Claridad para explicar por qué el sistema recomienda cada decisión.